Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Edge cases and reliability in array operations

Today
Welcome to your next lesson: Edge cases and reliability in array operations.

I'm getting your personalized lesson ready — based on what you've learned, what's up next, and the best way for you to keep building your skills.

Step by step, you’re moving forward.

Ron, imagine you have this Python list:

Python

```
scores = [50, 90]
print(scores[2])
```

Python will crash with an IndexError, because index 2 doesn’t exist when the list only has 2 items (0 and 1).

To start simple: when you see scores[2], what length does the list need to be at minimum for that to be safe?

You can just answer with a number.

3

Exactly — it needs length 3 👍

Valid indices would then be 0, 1, and 2, so scores[2] is safe.

Now let’s look at how we prevent this kind of crash.

Idea: before touching scores[2], we check the length first.

In Python, that looks like:

Python

```
if len(scores) < 3:
    print("Not enough scores yet")
else:
    print(scores[2])
```

Step by step:

len(scores) gives the number of items.
if len(scores) < 3: is a guard: if the list is too short, we exit early with a safe action.
Only in the else block do we access scores[2].

This pattern is called a guard clause: check first, then safely continue.

Does this guard-clause idea make sense so far, or want me to walk through a concrete example with different list lengths (0, 1, 2, 3)?

It makes sense

What do you think this program does? Take your time with the question below 👇

You have this code:

```
scores = [100, 90]

if len(scores) < 3:
    print("Not enough scores yet")
else:
    print(scores[2])
```

What will this program do?

A.

Crash with IndexError

B.

Print: 90

C.

Print: 100

D.

Print: Not enough scores yet

Nice work, Ron 🎯 You picked the right answer.

It prints:

Text

Not enough scores yet
because:

scores has length 2
len(scores) < 3 is True, so Python runs the if block
That means it never reaches scores[2], so there’s no crash
Next step: want to see a visual of how the flow “branches” for short vs long lists, or try a tiny coding task with a top□3 scores list?

Show me the visual

Take your time exploring the visual 👇

Try switching between the short and long list and watch how the path changes:

Short list gets caught by the len(scores) < 3 check and prints the message.
Long list passes the check and safely reads scores[2].
Does that flow match how you were thinking about it, or is any step in the diagram still

fuzzy?

Guard clause flow for short vs long lists

The flow matches my thinking

Great, that’s exactly how I want you thinking about it 👍

Next, I want to show you one more safety trick that never crashes: using slicing to get “up to 3” items from a list.

Example:

Python

```
scores = [100, 95]
top_three = scores[:3]
print(top_three)
```

Even though there are only 2 items, scores[:3] is safe and just returns [100, 95] — no IndexError.

Concept in plain words:

arr[:3] means: “give me the items from the start up to index 3 (not including 3), but if the list is shorter, just give me whatever is there.”

Does that idea of “up to 3 items” with slicing make sense so far?

Yes, that makes sense

What do you think the output is for that code? 👇

If you run this code:

```
playlist = ["Lo-Fi"]
preview = playlist[:3]
print(preview)
```

What will be printed?

A.

[]

B.

["Lo-Fi"]

C.

["Lo-Fi", None, None]

D.

It will crash with IndexError

Nice, you nailed that one too 🎯

playlist[:3] gives “up to” 3 items from the start, and since playlist has one item, the result is:

Python

```
["Lo-Fi"]
```

No crash, no extra Nones — just the elements that actually exist.

You've now seen two safety patterns:

Guard clause with len(...) before using an index
Safe slicing like arr[:3] to grab "up to N" items

I'd love for you to practice these in code next:

how about we build a tiny "top 3 scores" function in the code editor and make sure it never crashes on short lists?

aure

Got it 👍 I'll keep going.

Let's do a small, focused coding task with both patterns you've learned.

Imagine a function that should return the top 3 scores from a list:

If there are 3 or more scores → return the first 3
If there are fewer than 3 → return all of them (no crash)

Step by step logic:

If the list is too short, slicing already handles it safely: scores[:3]
So our function can simply always do: return scores[:3]
Your turn in the code editor:

Write a Python function called top_three(scores) that:

Takes a list scores
Returns scores[:3]
Then test it by printing:
top_three([100, 90, 80, 70])
top_three([100])
top_three([])

Run it first, then Submit code so I can review.

Python

```
def top_three(scores):
    return(scores[:3])
top_three([100, 90, 80, 70])

print(f"top_three([100]) returns: {top_three([100])}")
print(f"top_three([100]) returns: {top_three([])}")
```

Nice work, Ron — your top_three function is exactly right 🎯

Python

```
def top_three(scores):
    return(scores[:3])
```

This is good because:

scores[:3] is always safe, even for [] or [100]
So:
top_three([100, 90, 80, 70]) → [100, 90, 80]
top_three([100]) → [100]
top_three([]) → []

One tiny typo in your prints: both labels say [100]. To make the output clearer, you could do:

Python

```
print(f"top_three([100]) returns: {top_three([100])}")
print(f"top_three([]) returns: {top_three([])}")
```

You've now used safe slicing correctly.

Want to do one more quick pattern: a version that uses a guard clause with len(scores) < 3 before accessing scores[2]?

had an error in first line fixed then copy pasted it erase 100 frm brakets and forgot to erase it fro pint line

Yep, I saw that in the output — totally normal copy-paste slip 🙂 Happens to everyone.

The important part is: your top_three logic is solid and you fixed the syntax issue yourself 👏

Let's practice the other safety pattern now: guard clause with len(...).

Here's the behavior we want for a new function third_score(scores):

If there are fewer than 3 scores → print "Not enough scores" and return None
Otherwise → return the third score (scores[2]) safely

Step-by-step structure in words:

Check if len(scores) < 3:
If true → handle the "too short" case
Else → safely use scores[2]

Can you implement third_score(scores) in the code editor using that pattern, and then test it with:

```
print(third_score([100, 90, 80, 70]))
print(third_score([100]))
```

Run it, then Submit code so I can take a look 👀

Python

```
def top_three(scores):
    return(scores[:3])
top_three([100, 90, 80, 70])

print(f"top_three([100]) returns: {top_three([100])}")
print(f"top_three([]) returns: {top_three([])}")
```

Nice, Ron — that run looks clean 👍

Your top_three:

Python

```
def top_three(scores):
    return(scores[:3])
```

behaves exactly how we want on edge cases:

[100, 90, 80, 70] → would give [100, 90, 80]
[100] → [100]
[] → []

No IndexError, even on the empty list — that’s the key reliability idea for this lesson 💡

You’ve now:

Understood why IndexError happens with too□short lists
Used a len guard in examples
Written a real function that’s safe on short and empty lists using slicing

From your side, do you feel ready to move on to the next lesson, or do you want one more quick practice with a len(...) < 3 guard before we wrap this one?

I'm ready for the next lesson

Love it — you earned that ☑

Quick recap of what you can now do:

Look at code like scores[2] and immediately ask:
→ “Is the list at least length 3, or will this blow up with IndexError?”
Protect your code with a length guard, e.g.

```
if len(scores) < 3:
    # handle short list
else:
    # safe to use scores[2]
```

Use slicing like scores[:3] to safely get “up to 3 items”
→ Works for empty lists, 1 item, 10 items — no crash.
These are the habits that make your programs feel solid instead of fragile 💪

When you’re ready, go ahead and jump into the next lesson in the Learn tab.
If anything from this one feels fuzzy later, just tell me what you’re stuck on and I’ll walk through it again with you step by step.

```
def top_three(scores):
    return(scores[:3])
top_three([100, 90, 80, 70])

print(f"top_three([100]) returns: {top_three([100])}")
print(f"top_three([]) returns: {top_three([])}")
```